

01

thread_01.c

코드 완성

</01>	create
</02>	&tid, NULL, magic_box, (void*)(intptr_t)10
</03>	join
</04>	tid, &new_number

출력 결과

```
thread_01.c thread_02.c thread_03.c thread_04.c
kmg@2018320184:~/shared_folder/thread$ gcc -o thread_01 thread_01.c
kmg@2018320184:~/shared_folder/thread$ ./thread_01
Hey magic box, multiply 10 by 6
multiplying 10 by 6...
the new number is 60
kmg@2018320184:~/shared_folder/thread$
```

02

thread_02.c

코드 완성

</01>	exit
</02>	create
</03>	&tids[i], NULL, worker, &main_static
</04>	join
</05>	tids[i]

출력 결과

```
the new number is 60
kmg@2018320184:~/shared_folder/thread$ gcc -o thread_02 thread_02.c
kmg@2018320184:~/shared_folder/thread$ ./thread_02
global      main      thread      thread-static
0x60c6b070c014 0x60c6b070925c (nil)      (nil)
0x60c6b070c014 0x60c6b070c01c 0x78a4eaffeeb4 0x60c6b070c018
0x60c6b070c014 0x60c6b070c01c 0x78a4ea7fdeb4 0x60c6b070c018
0x60c6b070c014 0x60c6b070c01c 0x78a4e9ffceb4 0x60c6b070c018
kmg@2018320184:~/shared_folder/thread$
```

03

thread_03.c

코드 완성

</01>	exit
</02>	void*
</03>	create
</04>	&tids[i], NULL, worker, NULL
</05>	join
</06>	tids[i], &progress

출력 결과

```
0x0000000000000000 0x0000000000000000 0x0000000000000000 0x0000000000000000
kmg@2018320184:~/shared_folder/thread$ gcc -o thread_03 thread_03.c
kmg@2018320184:~/shared_folder/thread$ ./thread_03
855218
expected: 1000000
result: 855219
kmg@2018320184:~/shared_folder/thread$
```

04

thread_04.c

코드 완성

</01>	mutex_lock
</02>	&lock
</03>	mutex_unlock
</04>	&lock
</05>	create
</06>	&tids[i], NULL, worker, NULL
</07>	join
</08>	tids[i], &progress

출력 결과

```
result: 855219
kmg@2018320184:~/shared_folder/thread$ gcc -o thread_04 thread_04.c
kmg@2018320184:~/shared_folder/thread$ ./thread_04
973734
expected: 1000000
result: 1000000
kmg@2018320184:~/shared_folder/thread$
```

05

thread_05.c

코드

```
1  #include <stdio.h>
2  #include <stdatomic.h>
3  #include <unistd.h>
4  #include <pthread.h>
5  #include <sys/wait.h>
6  #include <stdint.h>
7  #define NUM_TASKS 3
8  #define SPREADING 2
9
10 static _Atomic int cnt_task = NUM_TASKS;
11
12 void spread_words() {
13     sleep(SPREADING);
14     printf("[subordinate] spreading words...\n");
15     cnt_task--;
16 }
17
18 void* subordinate(void* arg) {
19     sleep(SPREADING);
20     printf("[subordinate] as you wish\n");
21     for(int i = 0; i < 3; i++) {
22         spread_words();
23     }
24
25     pthread_exit(NULL); //hint1: Finish up the threads
26 }
27
28 void* king(void* arg) {
29     pthread_t tid;
30     int status;
31     printf("spread the words ");
32
33     //hint2: You should start your subordinate here
34     status = pthread_create(&tid, NULL, subordinate, NULL);
35     if (status != 0) {
36         printf("error");
37         pthread_exit(NULL);
38     }
39
40     pthread_detach(tid);
41
42     printf("that I am king!\n");
43     pthread_exit(NULL);
44 }
45
46 int main(int argc, char* argv[]) {
47     pthread_t tid;
48     int status;
49
50     status = pthread_create(&tid, NULL, king, NULL);
51     if (status != 0) {
52         printf("error");
53         return -1;
54     }
55     pthread_join(tid, NULL);
56
57     //hint3: Make the main thread wait for the subordinate to finish
58     //hint3: But cannot use pthread_join because subordinate thread is detached
59     //hint3: Think busy waiting
60     while (cnt_task > 0) {
61         // busy waiting
62     }
63
64     printf("The words have been spread...\n");
65     return 0;
66 }
```

line12~16: spread_words() – subordinate가 실제 작업 하나를 수행

line13: sleep(SPREADING)을 통해 작업 시간이 걸리는 상황을 정의

line14: 작업 수행 메시지 출력

line15: 남은 작업 개수 1 감소

line18~26: subordinate thread가 실행하는 함수

line19~20: subordinate thread 시작 후 잠시 대기. 이후 [subordinate] as you wish 출력

line21~23: 반복문에서 spread_words()를 3번 호출. 따라서 subordinate thread는 총 3개 작업 수행

line25: (코드 추가 부분) pthread_exit(NULL)은 subordinate thread를 정상 종료시킨다. 반환값은 따로 사용하지 않으므로 NULL로 설정하였다.

line28~44: king thread가 실행하는 함수

line31: 먼저 spread the words를 출력

line34: (코드 추가 부분) pthread_create()를 사용하여 subordinate thread를 생성 및 시작. tid의 주소, 실행할 함수(subordinate)를 인자로 설정. thread 속성과 thread 함수에 넘길 인자는 특별히 필요한 것이 없어 NULL로 설정.

line35~38: (코드 추가 부분) thread 생성 실패 시 에러 출력 후 king thread를 종료하는 예외 처리

line40: pthread_detach(tid) 호출. 이로 인해 subordinate thread는 detached thread가 되어 종료 시 자동으로 자원이 정리되지만, pthread_join()으로 기다릴 수 없다.

line42~43: that I am king!을 출력하고 king thread 종료

line46~55: main thread가 king thread를 생성하고 대기

line50: king thread 생성

line51~54: king thread 생성 실패 시의 예외 처리

line55: pthread_join(tid, NULL)을 호출하여 king thread가 끝날 때까지 대기

line57~62: busy waiting으로 subordinate 종료 대기

line60~62: (코드 추가 부분) subordinate thread는 detached 상태이므로 pthread_join()으로 기다릴 수 없다. 그래서 main thread는 공유 변수 cnt_task를 계속 확

인한다. subordinate가 spread_words()를 3번 실행하면 cnt_task는 3에서 0이 되고, 그 순간 while (cnt_task > 0) 조건이 false가 되어 반복문을 탈출한다.

line64~65: cnt_task가 0이 되면 subordinate의 모든 작업이 끝났다는 뜻이다. 따라서 line 64에서 The words have been spread...를 출력하고, line 65에서 프로그램을 정상 종료한다.

출력 결과

```
kmg@2018320184:~/shared_folder/thread$ gcc -o thread_05 thread_05.c
kmg@2018320184:~/shared_folder/thread$ ./thread_05
spread the words that I am king!
[subordinate] as you wish
[subordinate] spreading words...
[subordinate] spreading words...
[subordinate] spreading words...
The words have been spread...
kmg@2018320184:~/shared_folder/thread$
```

06

thread_06.c

코드

```
1  #include <stdio.h>
2  #include <stdatomic.h>
3  #include <unistd.h>
4  #include <pthread.h>
5  #include <sys/wait.h>
6  #include <stdint.h>
7  #define NUM_TASKS 3
8  #define SPREADING 2
9
10 static _Atomic int cnt_task = NUM_TASKS;
11 //hint1: Use cond to eliminate the busy waiting
12 //hint1: A mutex is required to use condition variables.
13 pthread_mutex_t lock;
14 pthread_cond_t cond;
15
16 void spread_words() {
17     sleep(SPREADING);
18     printf("[subordinate] spreading words...\n");
19
20     pthread_mutex_lock(&lock);
21     cnt_task--;
22     if (cnt_task == 0) {
23         //hint2: Wake up the main thread when cnt_task becomes 0.
24         pthread_cond_broadcast(&cond);
25     }
26     pthread_mutex_unlock(&lock);
27 }
28
29 void* subordinate(void* arg) {
30     sleep(SPREADING);
31     printf("[subordinate] as you wish\n");
32     for(int i = 0; i < 3; i++) {
33         spread_words();
34     }
35     //hint3: Finish up the threads
36     pthread_exit(NULL);
37 }
38
39 void* king(void* arg) {
40     pthread_t tid;
41     int status;
42     printf("spread the words ");
43
44     //hint4: You should start your subordinate here
45     status = pthread_create(&tid, NULL, subordinate, NULL);
46     if (status != 0) {
47         printf("error");
48         pthread_exit(NULL);
49     }
50
51     pthread_detach(tid);
52
53     printf("that I am king!\n");
54     pthread_exit(NULL);
55 }
56
57 int main(int argc, char* argv[]) {
58     pthread_t tid;
59     int status;
60     //hint5: Initialize the mutex and condition variable before use.
61     pthread_mutex_init(&lock, NULL);
62     pthread_cond_init(&cond, NULL);
63
64     status = pthread_create(&tid, NULL, king, NULL);
65     if (status != 0) {
66         printf("error");
67         return -1;
68     }
69     pthread_join(tid, NULL);
70
71     //hint6: Eliminate busy waiting by using a mutex and condition variable.
72     //hint6: This solution should be an upgraded version of thread_05.c.
73     pthread_mutex_lock(&lock);
74     while (cnt_task > 0) {
75         pthread_cond_wait(&cond, &lock);
76     }
77     pthread_mutex_unlock(&lock);
78
79     //hint7: You have to clean up mutex and cond
80     pthread_cond_destroy(&cond);
81     pthread_mutex_destroy(&lock);
82
83     printf("The words have been spread...\n");
84     return 0;
85 }
```

thread_05.c와 기능상 동일한 코드이기 때문에, thread_05.c와 비교했을 때의 차이점을 위주로 설명하겠다. thread_06.c는 cond를 사용하라는 것이 요점이다.

line13~14: mutex와 condition variable 정의 (코드 추가 부분)

condition variable은 mutex와 함께 사용해야 한다. main thread는 조건이 만족될 때까지 잠들고, subordinate thread가 조건이 만족되었음을 알려주면 깨어난다.

line16~27: spread_words() 함수

line20: mutex lock 획득. cnt_task는 main thread와 subordinate thread가 함께 접근하는 공유 변수이므로, 값을 변경할 때 mutex로 보호한다.

line21: 작업 하나가 끝났으므로 cnt_task--

line22~25: (코드 추가 부분) cnt_task == 0인지 확인. 0이면 모든 작업이 끝났다는 뜻이므로, pthread_cond_broadcast(&cond)를 호출하여 condition variable에서 기다리는 main thread를 깨운다.

line26: mutex lock 해제

line29~37: (코드 추가 부분) thread_05.c에서의 line18~26과 동일한 부분. exit을 사용하여 thread를 종료한다.

line39~55: king thread의 동작

line45~49: (코드 추가 부분) thread_05.c에서의 line34~38과 동일한 부분. subordinate thread를 생성한다.

그 외 다른 부분도 thread_05.c와 동일한 내용이다.

line57~69: main thread 실행 부분

line61~62: (코드 추가 부분) mutex와 condition variable을 초기화. 초기화하지 않은 mutex나 condition variable을 사용하면 정상 동작을 보장할 수 없다.

그 외 다른 부분은 thread_05.c와 동일한 내용이다.

line71~77: busy waiting 제거

line73: main thread가 mutex를 획득

line74: 아직 남은 작업이 있는지 확인

line75: pthread_cond_wait(&cond, &lock)은 main thread를 잠들게 한다. 이때 mutex는 자동으로 풀리고, 나중에 signal 또는 broadcast를 받고 깨어날 때 다시 mutex

를 획득한다. subordinate thread가 line 24에서 pthread_cond_broadcast(&cond)를 호출하면 main thread가 깨어난다. 깨어난 후 다시 while (cnt_task > 0) 조건을 확인하고, cnt_task가 0이면 반복문을 탈출한다. if가 아니라 while을 사용한 것은, condition variable에서는 깨어난 뒤 조건이 정말 만족되었는지 다시 확인하는 방식이 안전하기 때문이다.

line79~84: mutex와 condition variable 정리 후 최종 메시지 출력

line80~81: 사용이 끝난 condition variable과 mutex를 정리

그 외에는 thread_05.c와 마찬가지로 프로그램 종료 수순을 거친다.

출력 결과

```
The words have been spread...
kmg@2018320184:~/shared_folder/thread$ gcc -o thread_06 thread_06.c
kmg@2018320184:~/shared_folder/thread$ ./thread_06
spread the words that I am king!
[subordinate] as you wish
[subordinate] spreading words...
[subordinate] spreading words...
[subordinate] spreading words...
The words have been spread...
kmg@2018320184:~/shared_folder/thread$
```